

Beyond the Podium: A battle of dominance between F1's greatest

Huda Anum, Grishma Arun Kumar, Mehal Patel, Jolly Yan

March 2026

1 Introduction

Every year since 1950, Formula 1 has hosted around 20 races (Grand Prix) worldwide, with approximately 20 drivers competing in each. The journey to the checkered flag is deceptively complex: qualification day determines each driver's starting grid position, followed by a race lasting one to two hours where drivers push themselves and their machines to the limit, completing around 62 laps of a circuit totaling roughly 305 kilometers at speeds exceeding 370 km/h. Only the top 10 (more or less) finishers receive championship points, with the winner taking the largest share.

The problem we aim to address is how to use a graph structure to measure the relative dominance or competitiveness of Formula 1 drivers. At first glance, one might assume that the driver with the most wins or championships is simply "the best." However, F1 is not that straightforward; it's a larger-scale version of robotics or a hackathon, where multiple interconnected factors contribute to success.

The challenge is when we consider that drivers rarely remain static in their careers. They switch teams or, more significantly, the technology itself undergoes massive evolution approximately every five years due to regulatory changes.

Therefore, our approach restricts comparisons only to drivers who have raced against each other in the same race. By taking these direct head-to-head race statistics and aggregating them across their shared racing history, we can build a meaningful picture of relative performance. If Driver A consistently finishes ahead of Driver B in the same race under identical conditions (same car performance era, same track, same weather), we can establish a direct relationship. We define "finishing ahead" by calculating the percentage of shared races where one driver outperformed the other, allowing us to use the data on a consistent scale of 0-100% for the Y-axis of our Bokeh visualization. Furthermore, there are some races that are exactly half of the Grand Prix. These races are known as "sprint races". To ensure that our comparison metrics mathematically make sense, we implemented a weighted scoring system where points from Sprint races are multiplied by 0.5 to account for their shorter distance and reduced impact relative to a full Grand Prix. By averaging these encounters across multiple races and potentially multiple seasons, we construct a statistical foundation for comparison that controls for the variables that make F1 so complex.

We wanted to create an objective, data-driven approach that lets the head-to-head results speak for themselves.

2 Dataset Description

For this project, we utilized the Formula 1 World Championship (1950-2024) dataset, which provides a comprehensive historical record of all Formula 1 competitive seasons.

- **Source:** The dataset was obtained through Kaggle.
(<https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020>)
- **Format:** The data is provided in CSV format. These were the specific files of the dataset File that we used:
 - drivers.csv
 - races.csv
 - results.csv: represents each driver's specific result for a specific race

– sprint_results.csv

For all the files, we removed all data from before 2017 in order to make our project recent and relevant.

For results.csv, positionOrder was renamed to positionOrderUse. This was our primary metric for determining who "finished ahead". We also renamed the rank column (which represents the order of fastest laps) to fastestLapOrder to make its purpose clearer for computations. Additionally, we removed several columns that weren't necessary for the graph logic:

- constructorID
- positionText
- time
- milliseconds
- fastestLap
- fastestLapSpeed

For sprint_results.csv, positionOrder was also renamed to positionOrderUse for consistency with the results.csv file. We removed the columns:

- constructorID
- positionText
- time
- milliseconds
- fastestLap

3 Computational Overview

This project implements a data-driven framework to evaluate Formula 1 driver dominance by transforming historical race results into a multi-layered relationship network. By shifting from raw statistics to a graph-based model, we move beyond "podium counts" to analyze performance within the context of direct competition.

Data to represent domain: Graphs As outlined in our proposal, the core of the project is a custom Graph data structure (found in entities.py) that models the F1 domain.

- **Vertices (Driver objects):** Represent individual racers. Each driver stores a past_races dictionary mapping race IDs to RaceData instances. This allows us to maintain a history for each driver from 2017 to 2024.
 - **Race objects:** Represent the competitive events. These instances manage the list of drivers present at a specific circuit, facilitating the connection logic between racers
- **Edges (Competitive Relationships):** An edge exists between two drivers if they have competed in the same race. The final implementation used a weighted dictionary (neighbours) to store the performance gap between rivals, where the weight represents the average difference in their calculated "skill scores".
- **Dataset Integration:** We processed six CSV files (drivers.csv, races.csv, results.csv, sprint_results.csv, circuits.csv, status.csv). The logic in load_data.py performs an aggregation, joining these files into a graph structure.

Major Computations & Algorithms

- **Calculating Dominance (calculate_one_race):** As planned in the proposal, we implemented a custom scoring algorithm to normalize driver skill. The formula calculates a points-based score for each session.
 - **Base score:** Inversely proportional to the finishing position $((\text{TotalDrivers} - \text{Position}) \times \text{multiplier})$

- **Grid Gain:** Rewards drivers who overtake opponents (Difference between grid and position)
- **Bonuses:** Additional points for podium finishes (top 3) and setting the fastest lap
- **Sprint normalization:** Sprint race scores are multiplied by 0.5 to account for their shorter distance and lower stakes.

- **Head-to-Head Logic** To compare two drivers accurately, the program performs a set intersection on their race histories. This ensures the comparison is "fair" and makes sure that it only considers events where both drivers were competing in the same race. This isolates driver skill from varying car performance across different seasons. Returns a dictionary whose keys will be displayed as x-axis titles in Bokeh and values as tuples for driver A and driver B stats comparison to one another.

Visualizations and Reporting We utilized two distinct visualization strategies to present our computational results:

1. Network Relationship Graphs Using **NetworkX** (`visualization_network.py`):

- A macro-view using `nx.Graph` to show era-based clusters
- These graphs were created interactively, allowing users to hover over nodes to see driver names and codenames (e.g., "HAM", "VER")

2. Comparative Performance Dashboard Using **Bokeh** (`visualization_bokeh.py`):

Following the proposal's plan for a nested colourmapped chart, we used Bokeh to generate grouped bar charts.

- The x-axis is categorized by metrics (Win%, Podium%, etc), with distinct bars for each driver.
x-axis titles:
 - **# of wins:** The percentage of shared race sessions won by each driver individually
 - **# podium finishes:** The percentage of shared sessions where a driver finished in the top three positions
 - **# of times each driver finished ahead of the others:** The percentage of the number of times the final position of one driver was ahead of the other
 - **avg change in position:** The mean difference between a driver's starting position and their final finishing position across all shared sessions
 - **# of fastest laps amongst each other:** The percentage of shared sessions where one driver recorded a faster lap time than their rival
- We implemented a `HoverTool` and `factor_cmap` to ensure consistent colour-coding (e.g., Driver A is always dark blue) across all categories, making the strengths and weaknesses for each driver immediately visible.

Use of New Python Libraries (which need to be installed)

- **NetworkX:** Used to visualize the graph and node positioning (specifically the `spring_layout`)
- **Plotly:** Used to transform the graph coordinates into an interactive tool to enhance the user interface
- **Bokeh:** Used to visualize categorical data as bar charts for statistical comparison

4 Obtaining Datasets and Running Your Program

Libraries to install under `requirements.txt` and datasets are already added to the zip file

When the program is initially run, the following initial program load will be seen:

```

----- Beyond the Podium: A battle of dominance between F1's greatest -----
* * * * *
* * * * *

Every year since 1950, Formula 1 has hosted around 20 races (Grand Prix) worldwide, with approximately 20 drivers competing in each

Our program helps to measure the relative dominance of Formula 1 drivers.

```

Figure 1: Initial load

The user will then be prompted to enter an input from either '(A)', '(B)', or 'exit'. By entering '(A)', a NetworkX visualization graph will be displayed showing a graph of all the possible racers the users can compare. By hovering over the nodes, the name of the racer will appear

```

Our program helps to measure the relative dominance of Formula 1 drivers.
You can choose from the following commands:
- Show all drivers (A)
- Enter driver 1 to compare (B)
- Exit

Enter action: (A)

```

Figure 2: Pick (A)

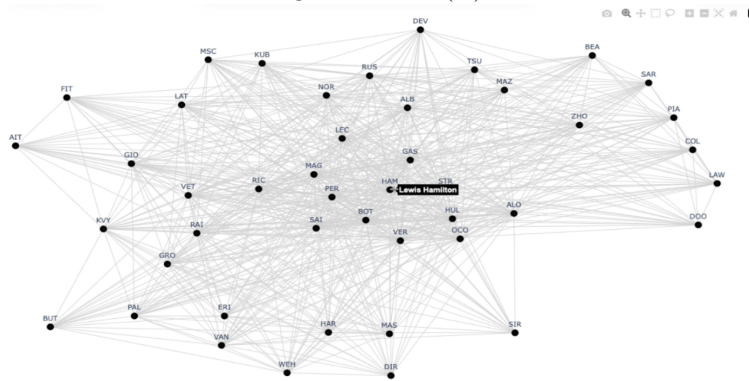


Figure 3: All drivers graph

By entering option ‘(B)’, the user will then be prompted to enter the name or racer code of the first racer they would like to compare. Once the driver code or name is entered, another NetworkX graph visualization will be displayed. This displays all the racers who have previously raced with the entered racer, allowing the user to choose their second racer. After the display, the user will be prompted to enter the code or name of the second racer. Once entered, a result summary of the comparison will be produced between the two racers, and a Bokeh bar graph will be displayed showing a visual comparison of the two racers. At any time, the user can enter ‘exit’ to terminate the program.

```
Enter action: (b)

Enter name of the first driver you would like to compare: HAM

Enter name of the second driver you would like to compare (NOTE: Cannot be the same as driver 1): |
```

Figure 4: Pick a driver using codename or full name

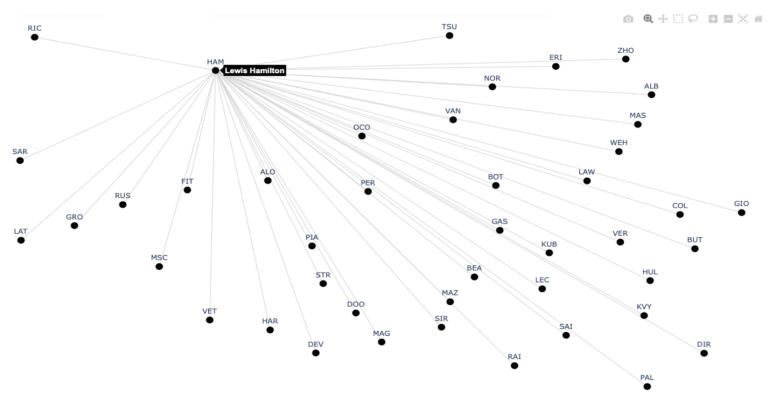


Figure 5: Graph of all Lewis Hamilton connections

```
Enter name of the first driver you would like to compare (You may enter the full name or driver code name): HAM
Enter name of the second driver you would like to compare (You may enter the full name or driver code name)(NOTE: Cannot be the same as driver 1): RUS
=====
HEAD-TO-HEAD STATS: 126 shared races
=====
Metric | Lewis Hamilton | George Russell
-----|-----|-----
# of wins | 25.4% | 2.4%
# podium finishes | 54.8% | 11.9%
# of times each driver finished ahead | 72.2% | 27.8%
avg change in position | 9.3% | -6.4%
# of fastest laps amongst each other | 76.2% | 22.2%
=====
```

Figure 6: Data shown on console for

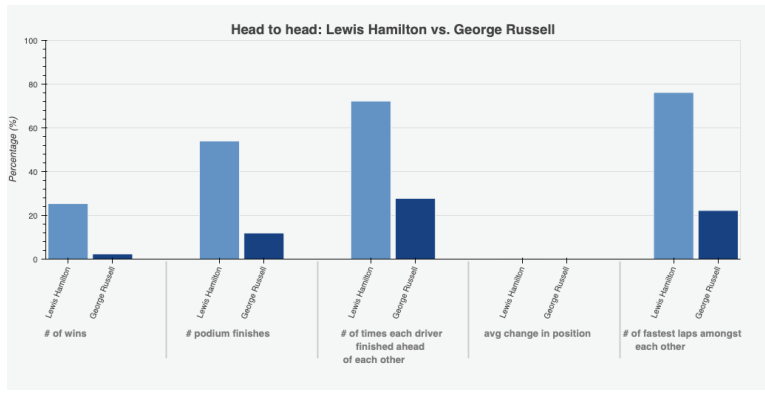


Figure 7: Bokeh comparison of Lewis Hamilton and George Russell

```

=====
You can choose from the following commands:
- Show all drivers (A)
- Enter driver 1 to compare (B)
- Exit

Enter action: exit
Thank you for using our project

```

Figure 8: Exiting the program

5 Changes From Our Proposal

However, while the final project has met the objectives set in the original plan, which is to compare the Formula 1 drivers using head-to-head comparisons, certain aspects of the project have evolved from the original plan. For instance, the data structure of the project has evolved from the use of lists to the use of dictionaries, which map IDs to objects. This has made the project more efficient, especially when dealing with large amounts of data.

In addition, we were able to address the problem of data ranges when visualizing the results of the project. In the initial plan, the project would compare the data in different numerical ranges, which made it difficult to create a uniform y-axis when creating the graphs of the results of the project. In order to solve the problem, we were able to convert all the data in the project to percentages by dividing the data by the total number of shared races among the drivers.

Another point to consider was the idea of engines. In our project proposal, our TA advised us to showcase the drivers as connections who were equipped with the same engine. However, given that the drivers are either teammates or belong to relatively few teams, this would significantly restrict our data. Because there would be fewer edges, the graph would appear less connected, which in turn makes our comparison less complex.

Furthermore, there are some races that are exactly half of the Grand Prix. These races are known as “sprint races”. This implies that the number of laps in a sprint race is half the number of laps in the Grand Prix. This phenomenon further extends to scoring regarding the points awarded to racers. In our implementation of `calculate_one_race`, if the specified session is a sprint race, then the points awarded to racers are divided by 2.

Although the initial idea was to include weighted and directed edges in the graph to show the dominance between the drivers, it was not included in the code due to certain problems faced in the project. The data structure we used was based on the driver IDs mapped to the objects. It was not feasible to include the weighted and directed edges in the code because it involved significant refactoring in the current structure of the code used in the head-to-head comparison system.

```
def create_single_driver_graph(graph: entities.Graph, driver_name: str) -> nx.DiGraph: 1 usage
    """
    """
    network_graph = nx.DiGraph()
    racer = graph.get_driver(driver_name)
    network_graph.add_node(racer.name)

    for driver in racer.neighbours:
        network_graph.add_node(driver.name)
        network_graph.add_edge(driver.name, racer.name, weight=racer.neighbours[driver])

    return network_graph
```

Figure 9: Code implementation failure

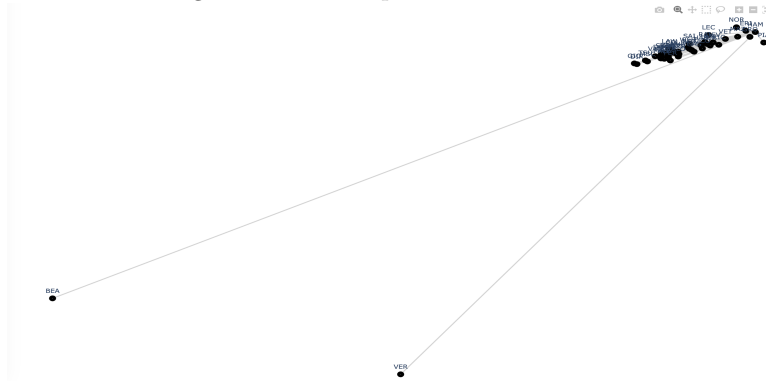


Figure 10: Weight graph look

6 Discussion Section

Our project aims to devise a data-driven approach to compare F1 drivers by looking at their head-to-head performance at races they both participated in, rather than looking at their overall race wins or titles. Overall, it can be said that our computational approach to the task has been successful. By creating a graph where edges represent races they both participated in, and storing race data within drivers represented as nodes, it is possible to generate statistics on their head-to-head performance, such as win percentage, podiums, fastest laps (number of times driver a had a faster lap compared to driver b), or average position change.

The results show that it is possible to devise a context-driven approach to compare drivers. Because we only compare drivers who competed in the same races, we avoid misleading cross-era comparisons (e.g., comparing drivers from completely different technological periods). Statistics, combined with the custom scoring system, allow the user to see not only who “won more” in races that they both participated in, but how they performed against each other. This is especially evident with the bar chart, which makes it easy to see the differences.

However, there are certain limitations that we have to consider in our analysis. To start with, we have to consider the limitations of the data set we are using. The data set we have may not account for certain factors, such as weather conditions and team strategies, apart from the fact that the driver did not finish the race. Moreover, it has to be considered that although we have designed the scoring system in a rather objective manner, it is still subjective to some degree. For example, the weights we have assigned to certain parameters, such as changes in position, fastest laps, and bonuses, may not be entirely objective and may have been different in certain cases. This is why we did not consider ‘consistency’ as a metric that we talked about in the proposal.

Moreover, from the point of view of implementation, we had to consider sprint races and normal races. This posed a certain degree of complexity in ensuring that we compared data of the same kind in different sessions. Moreover, it had to be ensured that we did not have issues of data not being provided in certain cases, such as in the case of drivers who did not finish the race.

7 References

“Everything You Need to Know about F1.” Formula 1® - the Official F1® Website, 2023, www.formula1.com/en/latest/article/drivers-teams-cars-circuits-and-more-everything-you-need-to-know-about.7iQfL3Rivf1comzdzqV5jwc.

“Graph.add_node — NetworkX 3.6.1 Documentation.” Networkx.org, 2025, networkx.org/documentation/stable/reference/classes/generated/networkx.Graph.add_node.html. Accessed 1 Apr. 2026.

“Hover Text and Formatting.” Plotly.com, plotly.com/python/hover-text-and-formatting/.

“Nested_colormapped.” Bokeh, 2016, docs.bokeh.org/en/latest/docs/examples/basic/bars/nested_colormapped.html. Accessed 31 Mar. 2026.

“Network.” Plotly.com, 2026, plotly.com/python/network-graphs/. Accessed 31 Mar. 2026.

Python Software Foundation. “Csv — CSV File Reading and Writing — Python 3.8.1 Documentation.” Python.org, 2020, docs.python.org/3/library/csv.html.

Python Software Foundation. “2. Lexical Analysis: Formatted String Literals.” Python.org, 2026, https://docs.python.org/3/reference/lexical_analysis.html.

“Tutorial — NetworkX 2.5 Documentation.” Networkx.org, networkx.org/documentation/stable/tutorial.html.

Vopani. “Formula 1 World Championship (1950 - 2024).” Kaggle.com, 2024, www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020/data. Accessed 31 Mar. 2026.

Note: *Parts of Assignment 3, “a3_part2_recommendations.py”, were referenced for visualization_network.py for “visualize_graph”, to create NetworkX Graph.*